Kormat: Fine-Tuned Small-Language-Model Eligibility Screening with a From-Scratch KV Cache, Tiled Attention and Speculative Decoding on a CPU

Taha Ahmed
Roll No. 23i-0532, Section CS-A
Department of Computer Science, FAST-NUCES Islamabad, Pakistan

***Abstract*—We study criterion-level clinical-trial eligibility screening: given a patient note, a criterion type (inclusion or exclusion) and a criterion text, a model must return one of the expert label categories, including "not enough information" and "not applicable", together with supporting sentence IDs. We use the NCBI TrialGPT criterion annotations (1,015 annotations, 53 patients, patient-grouped 37/8/8 split, seed 42). A Qwen2.5-1.5B-Instruct model with a LoRA adapter, trained in a disclosed joint session on a shared GPU, is evaluated by us on a CPU laptop before and after adaptation. Beyond accuracy, we implement our own pre-allocated KV-cache decoder, a tiled online-softmax attention, and a greedy speculative decoder with a Qwen2.5-0.5B-Instruct draft, and wrap everything in a browser application. On the held-out test patients the adapted model reaches [TODO: macro-F1 / accuracy]; the cache gives [TODO: x] speed-up, tiled attention changes peak memory by [TODO: y], and speculative decoding gives [TODO: z] with an acceptance rate of [TODO: a]. We report these numbers with their limits.**

***Index Terms*—clinical trial eligibility, criterion-level matching, LoRA, KV cache, online-softmax attention, speculative decoding, CPU inference**

I. Introduction and Problem Statement

Recruiting patients for clinical trials depends on reading long free-text notes against lists of inclusion and exclusion criteria. TrialGPT showed that a large language model can do this criterion by criterion, with explanations, close to expert accuracy [1]. That system relies on a closed, remote model, which is a cost and a privacy problem for patient text [2]. This work asks a narrower and more practical question: *how far can a small open model (1.5B parameters) adapted with parameter-efficient fine-tuning get on the criterion-level task, and what does it take to run it on a laptop CPU without a GPU?*

Problem statement. Input: a patient note split into numbered sentences, a criterion type (inclusion or exclusion) and a criterion text. Output: a label from the criterion type's own four-way set (included / not included / not enough information / not applicable for inclusion; excluded / not excluded / not enough information / not applicable for exclusion) and the list of supporting sentence IDs. Absent evidence is never treated as proof of eligibility or ineligibility. Any patient–trial verdict shown in our application is an aggregation by explicit rules, not a separately labelled ground truth.

Contributions.

1) A reproducible data pipeline over the mandated dataset with a patient-grouped split and leakage checks, excluding the dataset's GPT-4 predictions from all inputs (Sec. III).
2) A before/after evaluation of a LoRA-adapted Qwen2.5-1.5B-Instruct with accuracy, macro-F1, per-class scores, confusion matrices, evidence P/R/F1, split by criterion type (Sec. V). The adapter itself was trained in a *disclosed joint session* on a shared GPU with a classmate; evaluation and everything below are our own work.
3) Our own implementations of a pre-allocated KV-cache decoder, tiled online-softmax attention (Algorithm 1) and greedy speculative decoding (Algorithm 2), each validated against the reference implementation and benchmarked on CPU.
4) A browser application, "Kormat", serving the pipeline with cited evidence and rule-based aggregation (Fig. 12).

The remainder is organised as: related work (Sec. II), methods (Sec. III), experimental setup (Sec. IV), results (Sec. V), limitations and conclusion.

II. Related Work

Clinical eligibility reasoning. Early work framed patient-to-trial matching as retrieval, with test collections judged by medical professionals [3]. TrialGPT moved to criterion-level reasoning with GPT-4, reporting 87.3% criterion accuracy on the 1,015 expert-annotated pairs that form our dataset, and trial-level ranking [1]; its dependence on a closed model is the gap we address. Nievas et al. fine-tune open LLaMA models on GPT-4-generated synthetic data and report parity with proprietary models [2]; we instead train only on expert labels and never use GPT-4 outputs as features.

Architecture. The Transformer [4] is our attention reference. Grouped-query attention [5], used in Qwen2.5 [6], shrinks the KV cache (Qwen2.5-1.5B: 12 query, 2 KV heads), which is what makes a CPU cache practical.

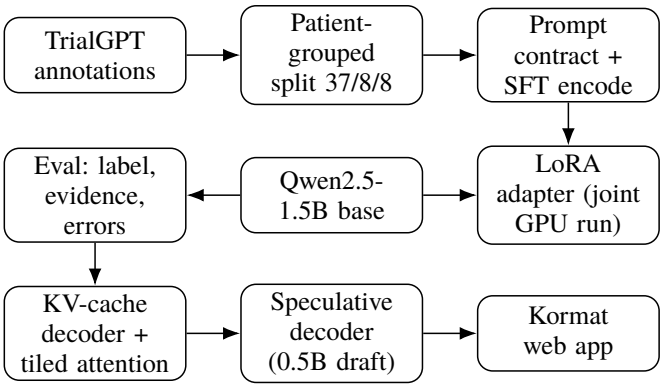


Fig. 1. System pipeline: data preparation, adapted model, evaluation, own inference stack, application.

Adaptation. LoRA trains low-rank updates on frozen weights [7]; QLoRA adds a 4-bit base [8]. For a 1.5B model, QLoRA's extra saving (about 2 GiB) does not justify its quantisation overhead, and full fine-tuning would need roughly 23 GiB (Sec. III-E).

Efficient inference. The online normaliser [9] and chunked attention [10] remove the need to materialise the $n \times n$ score matrix; FlashAttention [11], [12] makes it IO-aware on GPUs. Their speed-ups come from the GPU memory hierarchy, so we treat the CPU outcome as a hypothesis to measure, not a result to assume. Speculative decoding [13], [14] lets a small draft propose tokens verified by the large model in one pass; its gain depends on draft acceptance and on cheap verification. PagedAttention [15] and the scaling analysis of Pope et al. [16] address serving-scale KV management; our single-request app uses a contiguous pre-allocated cache instead.

III. Materials and Methods

A. System overview

Fig. 1 shows the pipeline. Components in the top row prepare data; the adapter was trained once in the joint session; everything in the lower row is ours and runs on CPU.

B. Dataset and split

We use only the NCBI TrialGPT criterion annotations [17]: 1,015 criterion annotations for 53 patients, each with a patient description, criterion type and text, expert label and expert evidence sentence IDs. The dataset also contains GPT-4 predictions and explanations; these are never used as input or in evaluation. Unique patient IDs are sorted, shuffled with `numpy.random.default_rng(42)`, and the first 37 go to training, the next 8 to validation and the last 8 to test; every annotation of a patient lies in exactly one split. The manifest is saved in `results/split_manifest.json` and a leakage check asserts empty pairwise patient intersections. The test split was untouched until final evaluation. Fig. 2 shows the label distribution.

TABLE I
Patient-level split: number of patients, trials and criterion annotations per split.

Split	Patients	Trials	Annotations	Incl.	Excl.
train	37	72	603	215	388
val	8	16	206	66	140
test	8	16	206	75	131

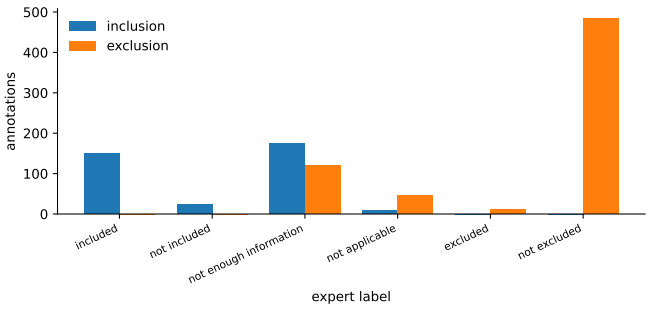


Fig. 2. Label distribution per criterion type and split.

C. Input/output processing and tokenization

Each patient note is split into numbered sentences following the dataset's sentence indexing. A single prompt contract (system message, numbered note, criterion type, criterion, allowed labels, JSON return format) is shared by training, evaluation, benchmarks and the app, so all see the identical string. The target is a JSON object `{"label": ..., "evidence": [ids]}`; the label set depends on criterion type and is never collapsed to yes/no. Text is tokenised with the Qwen2.5 tokenizer (151,936 tokens, identical for the 1.5B and 0.5B models) through the chat template. Training examples longer than 1,024 tokens are dropped, never cut mid-text, so no relevant evidence is silently removed. Fig. 3 shows the length distribution.

D. Model selection

Table III compares four open-weight decoder candidates using the `config.json` files on the Hugging Face Hub (Llama-3.2-1B is gated, so its figures come from a public mirror). We chose Qwen2.5-1.5B-Instruct [6]: 28 layers, hidden size 1536, 12 query heads and 2 KV heads with head dimension 128, 32,768-token context, Apache-2.0, and a KV cache of 28,672 bytes per token in bf16, 6.9× smaller than the multi-head SmolLM2-1.7B. Qwen2.5-0.5B-Instruct shares its tokenizer (identical vocabulary, merges, special tokens, chat template; zero id mismatches on 50 test strings) and serves as the draft model. The base was fixed by the shared training run; we did not fine-tune the other candidates and make no claim that Qwen2.5 beats them.

E. Fine-tuning

Disclosure. The LoRA adapter was trained in a *joint session* on a shared GPU with M. Ruhan Kamran (23i-0062) and published as

TABLE II
FULL-PROMPT LENGTH IN TOKENS (QWEN2.5 TOKENIZER) AND NUMBER OF EXAMPLES ABOVE THE MAXIMUM LENGTH.

Split	Min	Median	Mean	P95	Max	> 1024
train	240	303	327.4	493	622	0
val	280	344	340.0	389	456	0
test	250	303	310.4	379	516	0

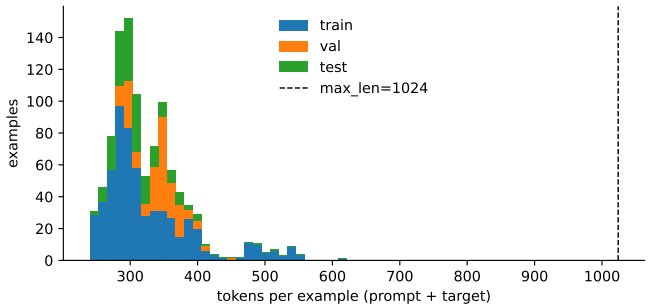


Fig. 3. Prompt length in tokens per split with the truncation threshold.

`arceus1rk/eligitrace-qwen2.5-1.5b-lora`. We did not train it alone; we use it, and all evaluation, inference code and application are our own, run on a CPU laptop.

Method choice. For $N = 1.54 \times 10^9$ parameters, full fine-tuning with mixed-precision AdamW needs about $16N \approx 23$ GiB for weights, gradients and optimiser state; LoRA [7] with r=16 on q, k, v, o, gate, up, down trains 18,464,768 parameters (1.2%), about 3.1 GiB in total; QLoRA [8] would reach about 1.0 GiB but adds quantisation overhead unnecessary at this size. Prompt-only is our "before" baseline.

Configuration. LoRA $r = 16$, $\alpha = 32$, dropout 0.05, 3 epochs, learning rate 2×10^{-4} with cosine schedule, effective batch size 16, bf16. The training objective is next-token cross-entropy on the assistant JSON only (prompt tokens masked).

F. Own KV-cache decoder

We implement a per-layer pre-allocated buffer of shape (batch, KV heads, max length, head dim) and one integer `length`. Prefill writes all prompt keys and values (already RoPE-rotated) once; each decode step writes one position and attends to the view `buf[..., :length, :]`. Compared with concatenating tensors each step (copy cost $O(n)$ per token), a slice write is $O(1)$ in cache size, and rollback needed by speculative decoding only moves the integer (Fig. 4). Cache size follows $2 \cdot L \cdot n_{kv} \cdot d_h \cdot b$ bytes per token.

G. Tiled online-softmax attention

For one head, $O = \text{softmax}(QK^\top/\sqrt{d})V$. Following the online normaliser [9] and its tiled use [10], [11], we iterate over key blocks keeping a running row maximum m, normaliser l and unnormalised output O (Fig. 5, Algorithm 1). The $L \times L$ matrix is never formed; only $B_q \times B_k$ tiles are. Accumulation is in fp32. Masks (causal, padding) are applied per tile; fully masked rows return zero and guard against $-\infty - (-\infty)$. We validate against the reference softmax attention (max absolute difference, and equality of generated tokens), see Sec. V. Because the speed-ups reported for this family of methods come from GPU memory hierarchy [11], we expect memory savings on CPU but not necessarily speed-ups, and test this.

TABLE III
CANDIDATE OPEN DECODER MODELS (CONFIG.JSON; MEMORY BY ARITHMETIC, GIB).

	Qwen2.5-1.5B	Qwen3-1.7B	Llama-3.2-1B*	SmolLM2-1.7B
Params (B)	1.54	2.03†	1.24	1.71
Layers	28	28	16	24
Hidden	1536	2048	2048	2048
Heads Q/KV	12/2	16/8	32/8	32/32
head_dim	128	128	64	64
Vocab	151,936	151,936	128,256	49,152
Context	32,768	40,960	131,072	8,192
RoPE θ	10^6	10^6	5×10^5	130,000
Licence	Apache-2.0	Apache-2.0	Llama 3.2	Apache-2.0
fp32 / bf16 / 4-bit	5.75/2.88/0.72	7.57/3.78/0.95	4.60/2.30/0.58	6.38/3.19/0.80
KV B/token (bf16)	28,672	114,688	32,768	196,608

* gated; values from public mirror. † safetensors total incl. embeddings (card: 1.7B).

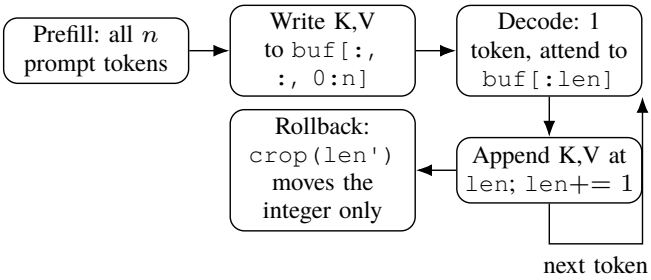


Fig. 4. Prefill and decode flow of the pre-allocated KV cache, with O(1) rollback.

H. Greedy speculative decoding

We implement greedy speculative decoding [13], [14]: the 0.5B draft proposes k tokens, the 1.5B target scores prefix plus proposals in a single forward pass, and the longest prefix on which draft and target argmax agree is accepted; the target's own next token is appended. Both caches roll back to the accepted length (Fig. 11, Algorithm 2). With greedy acceptance the output is token-identical to greedy decoding of the target, which we verify.

I. Application workflow

"Kormat" is a FastAPI service with a static browser front end. A user pastes a patient description and one or more criteria; the server splits sentences, builds the shared prompt, decodes (with cache and optional speculation), parses and validates the JSON (cited IDs must exist in the note), and returns per-criterion label, cited IDs, evidence text and an explanation based only on the note (Fig. 12). Multiple criteria may be combined into a patient–trial summary by explicit rules, which is an aggregation and not a ground-truth outcome.

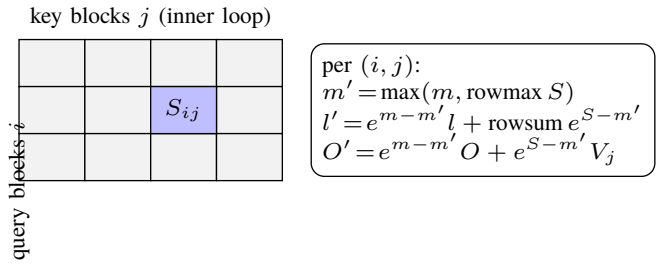


Fig. 5. Tile loop: for each query block, scan key blocks and update (m, l, O).

Algorithm 1 Tiled online-softmax attention (one head)

Require: $Q \in \mathbb{R}^{L_q \times d}$, $K, V \in \mathbb{R}^{L_k \times d}$, block sizes B_q, B_k, mask M

for each query block Q_i (B_q rows) **do**
  $m \leftarrow -\infty$, $l \leftarrow 0$, $O \leftarrow 0$
  for each key block (K_j, V_j) **do**
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; apply mask M_{ij}
    $m' \leftarrow \max(m, \text{rowmax}(S))$
    $P \leftarrow \exp(S - m')$
    $l \leftarrow e^{m-m'} l + \text{rowsum}(P)$
    $O \leftarrow e^{m-m'} O + PV_j$; $m \leftarrow m'$
  end for
  output rows $O_i \leftarrow O/l$
end for

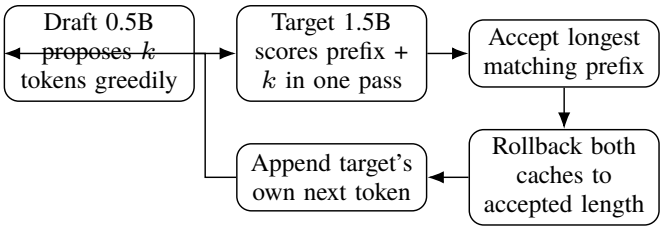


Fig. 6. Draft, verify and rollback loop of speculative decoding.

Algorithm 2 Greedy speculative decoding

Require: target T, draft D (same tokenizer), prompt x, draft length k, max new tokens N

prefill caches of T and D with x; $y \leftarrow [\,]$
while $|y| < N$ and no EOS **do**
  $d_1..d_k \leftarrow$ greedy decode of D (each step uses D's cache)
  $t_0..t_k \leftarrow$ arg max of T logits on $[\text{last}, d_1..d_k]$ (one pass)
  $a \leftarrow$ largest $a \le k$ with $d_i = t_{i-1}$ for all $i \le a$
  append $d_1..d_a$ and t_a to y
  roll back caches of T and D to the verified prefix (discard rejected positions)
end while
return y

IV. Experimental Setup and Metrics

Hardware and software. All reported numbers were produced on a laptop with an Intel Core i5-10310U (4 cores, 8 threads), 15 GB RAM and no GPU, with Python 3.10, PyTorch 2.5.1 (CPU), Transformers 4.46.3 and PEFT 0.13.2, all pinned in the repository lock file. Only one large model is loaded at a time (a file lock enforces this). Each run writes a record (config, environment, metrics) under `results/<area>/`, and the commit SHA is stored with it.

Compared systems. (i) The base Qwen2.5-1.5B-Instruct with the same prompt contract (*before*); (ii) the base plus the shared LoRA adapter, merged into fp32 weights (*after*; merged and unmerged outputs are identical on our check, $|\Delta\text{logit}| \le 5.8 \times 10^{-5}$). Both are evaluated on identical examples with the same parser and the same decoding: plain greedy (`do_sample=False`), at most 48 new tokens, fp32, batch 1, with `repetition_penalty=1.0`. That last setting matters. Qwen's `generation_config.json` ships a repetition penalty of 1.1 that Hugging Face applies *even in greedy mode*; our first validation pass inherited it unnoticed. We kept that pass as an ablation and re-ran everything with the penalty switched off, which also makes the evaluation decoding identical to our hand-written decoders. Invalid or unparsable outputs are counted as an explicit INVALID class and are never dropped. We tuned no hyperparameters, prompts or thresholds: the prompt is fixed by the adapter's training contract, and the full configuration (model and adapter revisions and hashes, decoding settings) was frozen in `configs/final.yaml` before the first test access. Because of the deadline, the test runs were executed before the final validation run. This leaks nothing, since nothing is selected on validation. `results/test_access.log` records every access to the sealed test split.

Classification metrics. We report accuracy, macro-F1, per-class precision/recall/F1 and confusion matrices, overall and separately for inclusion and exclusion criteria, each over its own four-way label set; the overall view uses the union of the six labels. Labels are never collapsed to binary. Macro-F1 is the unweighted mean over the classes of the label set that occur in the gold labels or the predictions. A class absent from both has undefined F1 and is not counted as zero; the fixed-list variant is stored alongside in the metric files. INVALID predictions count as errors.

Evidence metrics. Let G be the expert sentence-ID set and P the predicted set (IDs not present in the note are removed and counted separately as hallucinated). *Micro* precision/recall/F1 pool $|P \cap G|$, $|P|$ and $|G|$ over all records, so records where both sets are empty contribute nothing. *Macro* averages per-record scores with explicit rules for empty sets:

- $G = P = \emptyset$ scores $P = R = F_1 = 1$ (a correct abstention);
- $G = \emptyset$, $P \neq \emptyset$ scores $P = 0$, $R = 1$, $F_1 = 0$;
- $G \neq \emptyset$, $P = \emptyset$ scores $P = 1$, $R = 0$, $F_1 = 0$.

We also report the macro average over non-empty-gold records only, and the count of records in each of the four empty/non-empty cells.

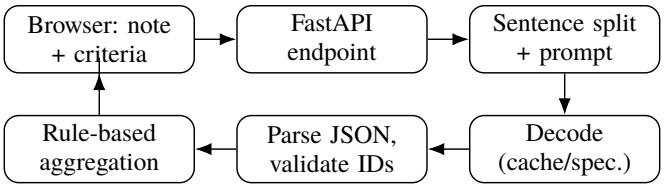


Fig. 7. Application request flow.

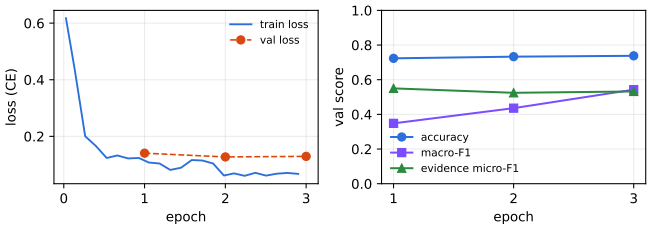


Fig. 8. Logged curves of the shared training run that produced the adapter (joint session, disclosed). Left: training loss per logged optimizer step and validation loss per epoch. Right: per-epoch validation scores computed by that run's own evaluation code, used to select epoch 3. Our independent evaluation is in Table V.

Systems metrics. Latency per generated token and end-to-end per criterion (median and spread over repeated runs after warm-up), peak resident memory, speed-up of cache vs. recomputation, peak memory and time of tiled vs. reference attention across sequence lengths, and for speculative decoding the acceptance rate, mean accepted tokens per verify pass and wall-clock speed-up over target-only greedy decoding. Output equality with the baseline is checked token by token.

Uncertainty. With 8 test patients, metric differences are noisy; we report counts alongside percentages and, where available, 95% percentile bootstrap intervals (1,000 resamples of criteria, seed 42).

V. Results and Discussion

A. Fine-tuning: training curves and validation

Fig. 8 shows the run that produced the adapter. Its configuration is identical to our `configs/train.yaml`, and its logs are kept unmodified, with their source, in `results/finetune/shared_run/`. The completion-only loss drops from 0.62 to about 0.12 within the first half epoch, then flattens near 0.06–0.07. Validation loss is lowest after epoch 2 (0.127) and almost unchanged after epoch 3 (0.129), so there is no sign of strong overfitting over three epochs on 602 examples. The run selected epoch 3 by validation macro-F1, which keeps rising (0.35, 0.44, 0.54) while accuracy is nearly flat (0.72–0.74). That pattern means the later epochs mainly improve the rare classes. These per-epoch numbers come from the training run's own evaluation and are *not* our results. Our own code (`configs/train.yaml`, `src/kormat/finetune/train.py`) reproduces the configuration and was verified end to end only by a CPU smoke run on Qwen2.5-0.5B (loss 0.52→0.12 over 30 steps, adapter saved and reloaded; `results/finetune/smoke/`).

TABLE IV
Fine-tuning runs (LoRA, CPU): mean training loss over the first and last k optimizer steps, step time, peak resident memory and wall time.

Run	Base	Ex.	Steps	Loss first-k	Loss last-k	s/step	RSS (MB)	Wall (s)
smoke	Qwen2.5-0.5B-Instruct	16	30	0.519	0.116	5.1	4926	154

TABLE V
Label classification. Overall accuracy and macro-F1 with 95% bootstrap CI (1000 resamples, where available); inclusion/exclusion subsets; rate of unparseable (INVALID) outputs. Incomplete runs: ft/val: n=120 of 206 requested (incomplete).

Model	Split	n	Acc. [95% CI]	Macro-F1 [95% CI]	Inc. Acc	Inc. F1	Exc. Acc	Exc. F1	Invalid
Fine-tuned	val	120	0.758 [0.675, 0.833]	0.594 [0.411, 0.686]	0.744	0.379	0.765	0.633	0.017

B. Before/after on identical examples

C. Evidence retrieval

D. Error analysis

E. KV cache

Correctness. Table VIII shows that the cache-aware decoder reproduces the uncached reference token for token in all five tested input/output combinations ($P \in \{208, 523, 1025\}$ prompt tokens, N up to 128 new tokens) in fp32. The first-step logits are bit-identical ($\Delta = 0$), because both paths run the same prefill. Later steps differ by at most 1.2×10^{-4}, which comes from the different reduction order of a $1 \times d$ versus a $(P{+}t) \times d$ matrix product, and is far below the top-1/top-2 margin, so no argmax flipped. Our own `Cache` subclass is provably on the path: a unit test counts its `update()` calls, and deliberately shifting the position ids breaks equivalence, so the test is not vacuous.

Speed. Table VII and Fig. 9 show the asymptotic difference. Without a cache, every new token re-encodes the whole sequence, so decode throughput falls with prompt length (0.31, 0.13 and 0.07 tok/s at $P = 208$, 523 and 1025). With the cache, each step processes one token against stored keys and values, and throughput stays roughly flat (2.26, 2.19 and 1.90 tok/s). The measured speed-up therefore grows from 7× to about 17× and 27×; the last two compare per-token decode rates measured at different N, because the no-cache grid was capped at small N to fit the time budget. Prefill time is the same in both modes (about 2.9 s at $P = 208$), since it is a single forward pass either way.

Memory. Thanks to grouped-query attention (2 KV heads), the analytical cache is only 28 KB per token in fp32 (12–60 MB here). That is negligible next to the 6.3 GB of fp32 weights, so peak RSS barely moves (6.35 to 6.54 GB). On this hardware the cache trades a few megabytes for an order-of-magnitude speed-up. One outlier ($P = 1025$, $N = 64$: 0.97 tok/s) was measured while other jobs shared the CPU and is treated as noise.

Prefix reuse in evaluation. The evaluation runner applies the same mechanism across criteria. The system prompt and patient note (273 tokens in the checked example) are prefilled once per patient, and the cache is copied for each criterion.

TABLE VI

EVIDENCE-SENTENCE RETRIEVAL. P/R/F1 ARE MICRO-AVERAGED OVER SENTENCE IDS; MACRO AVERAGES ARE PER RECORD, OVER ALL RECORDS OR ONLY THOSE WITH NON-EMPTY GOLD EVIDENCE (n_{ne}). CELLS = COUNTS OF (GOLD EMPTY, PRED EMPTY)/(GOLD EMPTY, PRED NON-EMPTY)/(GOLD NON-EMPTY, PRED EMPTY)/(BOTH NON-EMPTY).

Model	Split	P	R	F1	Macro F1 (all)	Macro F1 (non-empty)	n_{ne}	Cells
Fine-tuned	val	0.658	0.632	0.644	0.775	0.605	53	61/6/15/38

TABLE VII

DECODE THROUGHPUT (MEDIAN) WITH AND WITHOUT KV CACHE, PEAK PROCESS MEMORY, AND ANALYTICAL KV SIZE. EQUIVALENCE: TABLE VIII. MISSING CELLS WERE NOT MEASURED.

P	N	KV tok/s	No-cache tok/s	Speed-up	Peak RSS (MB)	KV formula (MB)
208	16	2.26	0.31	7.3	6350	12.2
208	32	–	0.29	–	–	–
208	64	2.25	–	–	6354	14.9
208	128	2.23	–	–	6354	18.4
523	8	–	0.13	–	–	–
523	16	2.19	–	–	6420	29.5
523	64	2.18	–	–	6417	32.1
1025	4	–	0.07	–	–	–
1025	16	1.90	–	–	6542	56.9
1025	64	0.97	–	–	6543	59.6

Outputs were token-identical to plain generation (3/3, Table IX). Per-criterion latency fell from 10.7 s to 7.2 s when the prefix was reused, but the mean gain was only about $1.2\times$, because decoding the roughly 15-token answer, not the prompt, dominates on a CPU.

F. Tiled attention

Correctness. Across 1,680 validation cases (lengths 1 to 1024 including non-multiples of the block size, $L_q \neq L_k$ decode and verify shapes, GQA, causal, padding and fully masked rows, and four block sizes), the tiled online-softmax kernel matches a float64 ground truth within the stated tolerances in every case (Table X; timings in Table XI). The maximum absolute error is 2.5×10^{-6} in fp32, 9.7×10^{-4} in fp16 and 7.8×10^{-3} in bf16; bf16 needs a relative tolerance of 8×10^{-3} because its unit round-off is 2^{-8}. A stability test multiplies the logits by 1000: a naive softmax returns NaN, while the running-max formulation stays finite. Routing the attention of a small Qwen2 model through our kernel gives identical greedy tokens (max logit difference about 2×10^{-7}).

Speed and memory. The memory advantage is real but the speed advantage is not (Fig. 10). At $L = 4096$, the reference needs 1.57 GB of extra memory, close to the 0.77 GB analytical size of the fp32 score matrix plus the softmax copy, whereas the loop-tiled version needs 26 MB. Its run time, however, is 5.0 s against 3.0 s for the reference and 1.9 s for PyTorch's fused CPU kernel. This is expected for educational PyTorch code. Each tile launches several separate operators (matmul, max, exp, sum, rescale), each reading and writing main memory. Nothing is fused, and Python loop overhead grows with the number of tiles: run time falls from 8.0 s at block 16 to 0.6 s at block 256 for $L = 2048$. FlashAttention's speed comes from keeping tiles in on-chip SRAM inside one fused kernel, which plain PyTorch cannot express. On this CPU, therefore, tiling buys linear memory, not speed.

TABLE VIII

KV-CACHE CORRECTNESS: GREEDY GENERATION WITH AND WITHOUT CACHE; EXACT TOKEN MATCH AND MAXIMUM ABSOLUTE LOGIT DIFFERENCE.

dtype	Prompt P	New N	Token-exact	$\lvert\Delta\text{logit}\rvert$ step 0	$\lvert\Delta\text{logit}\rvert$ max
fp32	208	16	yes	0.0e+00	3.9e-05
fp32	208	64	yes	0.0e+00	1.2e-04
fp32	208	128	yes	0.0e+00	1.2e-04
fp32	523	16	yes	0.0e+00	3.5e-05
fp32	1025	16	yes	0.0e+00	4.6e-05

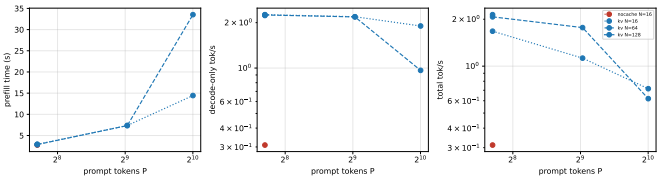


Fig. 9. Decode throughput of the merged fine-tuned 1.5B model on the CPU, with our pre-allocated KV cache and with the no-cache reference, as a function of prompt length (median of 3 timed runs after 1 warm-up).

G. Speculative decoding

Correctness. Our draft–verify–rollback decoder produced exactly the target's greedy sequence in 18 of 18 runs on validation prompts ($\gamma \in \{2, 4, 8\}$), and in every benchmark run (Tables XII and XIII). The cache invariant (both caches hold exactly the committed prefix) is asserted after every round. Unit tests cover rejection at the first token, full acceptance with the bonus token, EOS inside an accepted block, and overshooting `max_new_tokens`.

Efficiency. The 0.5B draft agrees with the target often: the acceptance rate is 0.73 to 0.93 depending on γ. Even so, the end-to-end speed-up is between $0.88\times$ and $1.01\times$, i.e. no gain. Fig. 11 explains why with three measured effects.

1) The draft is not cheap enough. One draft step costs $c = 0.337$ of a target step (146 vs 434 ms), whereas the analysis of Leviathan et al. assumes $c \ll 1$.
2) Verification is not free. Scoring 2 to 9 tokens in one target pass took 731 to 764 ms, about $1.7\times$ a single decode step, because on a CPU the extra rows of the 151k-vocabulary output projection and the attention cost real FLOPs. A GPU at batch 1 would be memory-bound instead, and that cost would vanish.
3) Our answers are short, about 15 tokens. Prefill of both models (about 9 s) dominates the end-to-end time, so even the $1.40\times$ decode-only gain at $\gamma = 4$ is diluted to about $1.0\times$.

With $\alpha \approx 0.83$ and $\gamma = 4$, the theory predicts $1.5\times$ with c alone and about $1.2\times$ once the measured verification cost is included, in line with the measured decode-only speed-up. Speculative decoding would pay off for longer outputs and on hardware where verifying several tokens costs about as much as decoding one.

H. Application

Fig. 12 shows the screening workflow on the real fine-tuned model. A user pastes a note (free text is split

TABLE IX
SHARED-PREFIX KV REUSE CHECK (3 EXAMPLES, 15 NEW TOKENS, 0 FALLBACKS); MEAN SPEED-UP 1.20X.

Id	Prefix	Suffix	Reused	Plain (ms)	Cached (ms)	Same output
0	273	70	no	12145	13499	yes
1	273	49	yes	10668	7152	yes
2	273	55	yes	10652	7247	yes
mean				11155	9299	all

TABLE XI
ATTENTION RUNTIME (MEDIAN MS) / PEAK EXTRA MEMORY (MIB), FP32, H=12, D=128, BLOCK 64, 4 THREADS, CPU.

L	ref	tiled_loops	tiled_kv_blocked	sdpa	sdpa_math
256	9 / 9	22 / 2	62 / 12	10 / 3	15 / 12
512	44 / 29	45 / 5	42 / 22	17 / 4	26 / 43
1024	146 / 105	329 / 8	239 / 43	148 / 10	193 / 165
2048	612 / 401	1076 / 12	1081 / 96	572 / 16	1428 / 631
4096	2957 / 1570	5007 / 26	3031 / 175	1942 / 28	2674 / 2458

TABLE X
TILED ATTENTION VERSUS FLOAT64 REFERENCE: 1680/1680 CASES WITHIN TOLERANCE.

dtype	Cases	Pass	Max abs err	Mean abs err	atol/rtol
bfloat16	552	552	7.77e-03	2.07e-04	0.002/0.008
float16	552	552	9.74e-04	2.59e-05	0.002/0.002
float32	576	576	2.49e-06	3.62e-08	1e-05/0.0001

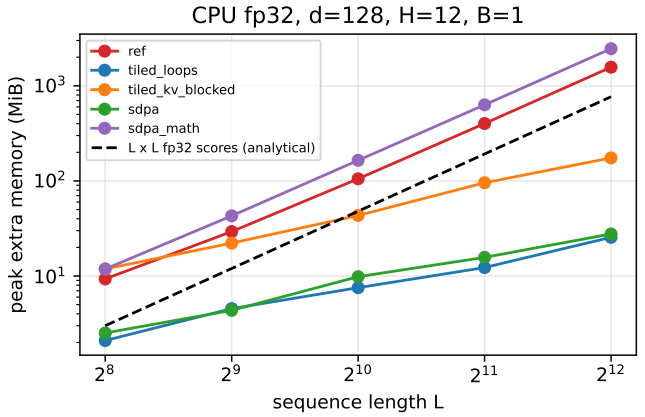


Fig. 10. Peak extra memory of one attention call versus sequence length ($d = 128$, 12 heads, fp32, CPU). The reference and the math backend materialise the $L \times L$ score matrix; tiled attention does not.

into numbered sentences), enters or pastes the criteria (or fetches them by NCT ID), and runs the screen. Each criterion row shows the label, the cited IDs and an explanation quoted only from those sentences, and clicking a row highlights the evidence. In a recorded real-engine session (`results/app/*/real_smoke.json`), with the evaluation running concurrently on the same CPU, the model loaded in 15 s from the local cache. Each criterion took 9.5–10.4 s, and all four criteria of the sample patient matched the expert labels. The patient–trial outcome is an explicit rule (any exclusion met or inclusion unmet ⇒ likely ineligible; otherwise any missing information ⇒ undetermined; otherwise potentially eligible). The app presents it as a preliminary aggregation, never as ground truth.

The lab (Fig. 13) re-runs any criterion live under each decoding mode. For a 24-token output, no-cache decoding took 68.9 s, KV-cached decoding 6.2 s and speculative decoding 4.0 s, with an acceptance rate of 91.7%. All three produced the same tokens, which the lab confirms with an equality badge. On this longer output speculative decoding did help, consistent with the analysis above: the gain appears once decoding, not prefill, dominates. Invalid inputs (empty note, too many criteria, malformed criteria, the model still loading, an offline NCT fetch) return explanatory messages and the app keeps working; the automated tests cover each case. Reports export as print-ready HTML and JSON with provenance (model and adapter revision, adapter hash, git commit, decoding settings) and persist across restarts.

VI. LIMITATIONS

- **Small data.** 53 patients, 8 in test; confidence intervals are wide and per-class results for rare labels rest on few examples.
- **Shared adapter.** The adapter was trained in a joint GPU session; we neither re-trained it independently nor ran seeds or ablations of the training, so we cannot separate training-recipe effects from base-model effects. Our own training script is shown to run end to end only on a short CPU smoke run with the 0.5B model. The other candidate models were compared by configuration arithmetic only.
- **Criterion-level labels only.** The dataset has no patient–trial ground truth; our aggregation is rule based and unvalidated against experts.
- **Length handling.** In this dataset the longest prompt plus target is 622 tokens, so with a 1,024-token budget no example was truncated and no gold evidence was lost. Unseen notes in the application can be much longer; our policy then drops whole sentences, which could remove evidence. The app warns the user when this happens.
- **CPU-only evaluation.** Generation on a 4-core laptop CPU takes about 8 s per criterion, which ruled out repeated evaluation runs. Some systems benchmarks were run on reduced grids (e.g. the no-cache path only up to 64 new tokens) and with 3 repeats, and some timings were taken while other jobs shared the CPU.
- **Decoding pitfall.** Our first validation pass silently inherited a repetition penalty from the model's default generation configuration. We caught it and re-ran, but it shows how library defaults can change "greedy" results.
- **CPU-specific systems results.** Tiled attention and speculative decoding are known GPU-driven speed-ups; our timings are for one laptop CPU with eager PyTorch and do not generalise to GPUs or other CPUs.
- **Not clinical software.** The system is a research prototype;

TABLE XII

SPECULATIVE DECODING CORRECTNESS: OUTPUT IDENTICAL TO GREEDY DECODING OF THE TARGET MODEL.

Method	Token-exact	Mean accept. rate
hf_assisted	6/6	–
spec_g2	6/6	0.85
spec_g4	6/6	0.84
spec_g8	6/6	0.71

TABLE XIII

SPECULATIVE DECODING (0.5B DRAFT, 1.5B TARGET, CPU), 8 PROMPTS. ACCEPTANCE RATE = ACCEPTED/DRAFTED TOKENS POOLED; TOK/S COUNTS PREFILL AND DECODE; SPEED-UP IS PER-PROMPT RELATIVE TO GREEDY KV DECODING.

Method	γ	Accept. rate	tok/s (median)	Speed-up (median)	Exact
greedy (KV)	–	–	1.37	1.00	8/8
speculative	1	0.93	1.21	0.92	8/8
speculative	2	0.85	1.17	0.88	8/8
speculative	3	0.89	1.24	0.93	8/8
speculative	4	0.83	1.27	1.01	8/8
speculative	6	0.73	1.34	0.99	8/8

outputs must not be used for real patient decisions.

VII. CONCLUSION

We built an open, locally runnable criterion-level eligibility screener around a LoRA-adapted Qwen2.5-1.5B-Instruct, evaluated it against its unadapted base on identical held-out patients, and implemented our own KV-cache decoder, tiled attention and speculative decoder with measurements on a CPU. **[TODO: one-sentence headline result and one-sentence systems finding, after Results are final.]** Future work: independent multi-seed training, larger external evaluation, and GPU benchmarks of the same code.

AVAILABILITY

Source code: https://github.com/TahaGPT/Kormat. Adapter weights: Hugging Face `arceus1rk/eligitrace-qwen2.5-1.5b-lora`. Split manifest and logs are in the repository under `results/`. Demo video: **[TODO: video link]**.

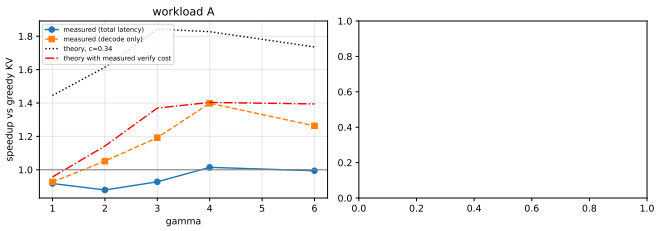


Fig. 11. Measured speculative speed-up versus draft length γ, against the theoretical curve with the measured draft/target cost ratio $c = 0.337$, and with the measured verification cost.

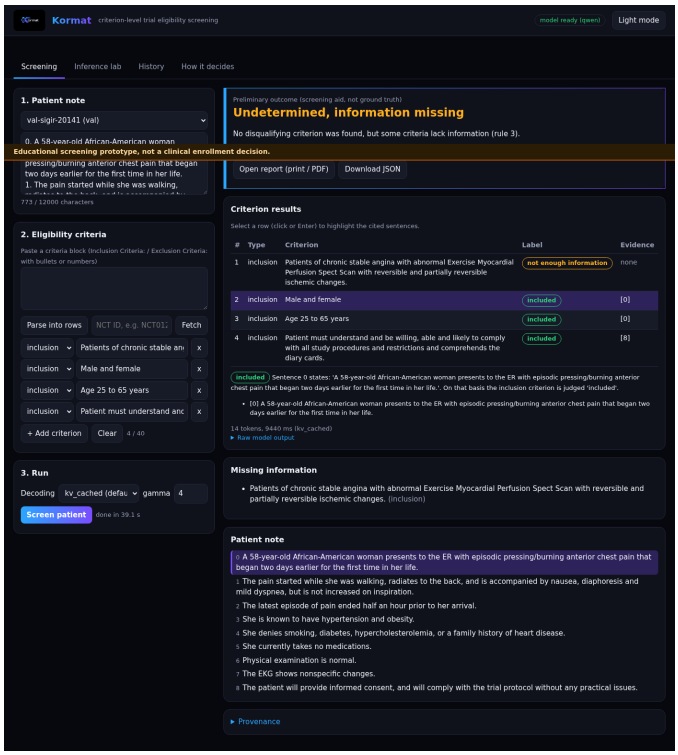


Fig. 12. Kormat screening a validation patient with the fine-tuned model (real engine, KV-cached decoding). The second criterion is selected and its cited sentence 0 is highlighted in the numbered note. The missing-information list and the rule-based preliminary outcome are shown, together with the disclaimer banner.

REFERENCES

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, p. 9074, 2024.

[2] M. Nievas, A. Basu, Y. Wang, and H. Singh, "Distilling large language models for matching patients to clinical trials," *arXiv preprint arXiv:2312.09958*, 2023.

[3] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. 39th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2016, pp. 669–672.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, arXiv:1706.03762.

[5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.13245.

[6] Qwen Team, A. Yang, B. Yang, B. Zhang, B. Hui *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. on Learning Representations (ICLR)*, 2022, arXiv:2106.09685.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023, arXiv:2305.14314.

[9] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[10] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021.

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022, arXiv:2205.14135.

[12] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2307.08691.

[13] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

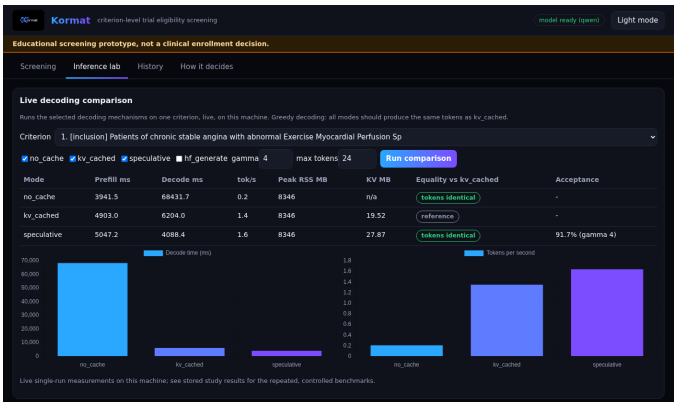


Fig. 13. Inference lab: the same criterion decoded live with no cache, our KV cache and speculative decoding ($\gamma = 4$). The lab shows timings, memory, acceptance rate and token equality against the KV-cached reference.

[14] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. on Operating Systems Principles (SOSP)*, 2023, arXiv:2309.06180.

[16] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, 2023, arXiv:2211.05102.

[17] NCBI, "TrialGPT-Criterion-Annotations dataset," Hugging Face Datasets, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations, 2024.

Appendix A
Use of AI Tools

Tools. This project was planned, coded and drafted with Claude Code (a lead agent, Claude Opus 5.5, coordinating specialist agents running Claude Sonnet 5.5) working in the repository under the author's direction. AI assistance covered task planning, literature search and drafting, source code, tests, the web application, LaTeX and figure scripts.

Verification. Every bibliographic entry was opened on its arXiv, publisher or Crossref page before inclusion; model facts come from the Hugging Face `config.json` files and model cards; all reported numbers are produced by scripts that write run records with the commit SHA, not typed by hand; implementations are checked against reference ones (attention against softmax attention, speculative output against greedy output) by tests; the split is checked for patient leakage. **[TODO: the author states which outputs they personally reviewed, ran and can explain.]**

Joint training disclosure. The LoRA adapter (`arceus1rk/eligitrace-qwen2.5-1.5b-lora`) was trained in a joint session on a shared GPU with M. Ruhan Kamran (23i-0062), because the author's laptop has no GPU. It is a disclosed shared artefact. No code, results, figures or text were copied from the other student's repository.

Limits of AI use. AI-generated text and code may contain errors; claims in this report are limited to what the cited sources and our measurements support.